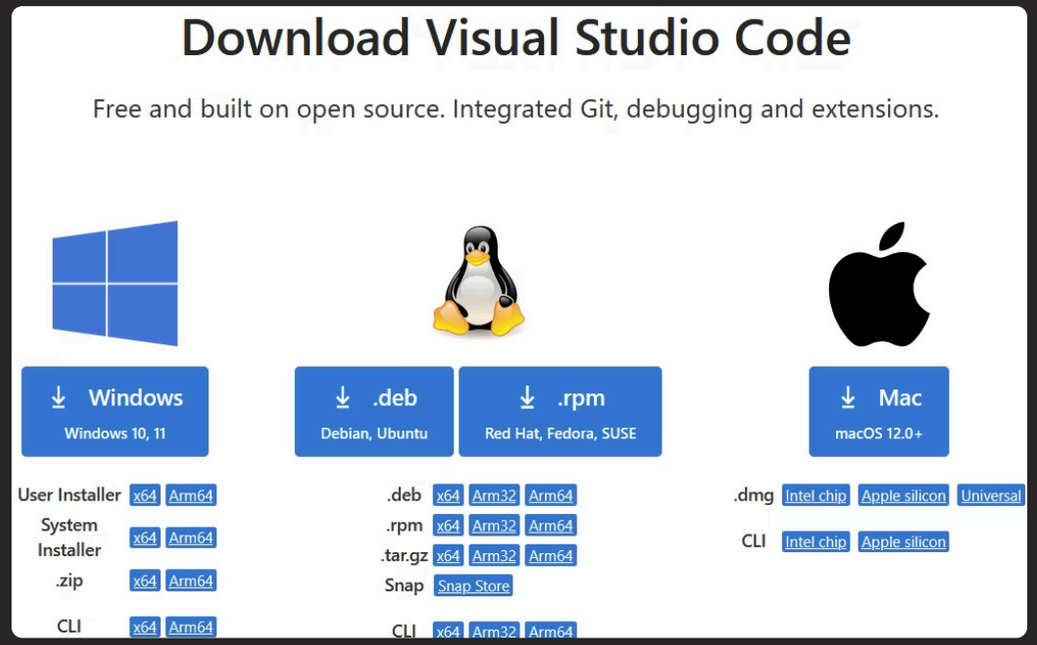
PROS

- Sensores que miden
- Control que corrige
- Robots que ganan

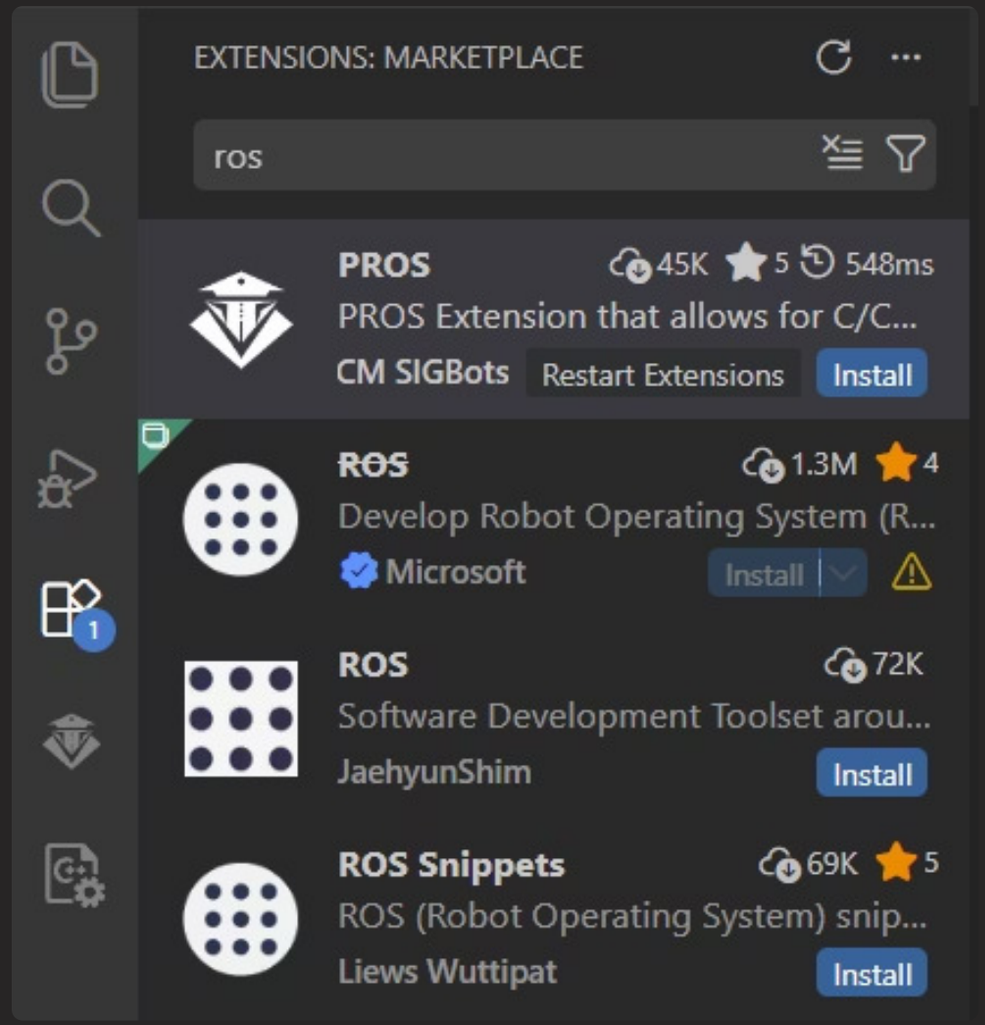
Made with GAMMA

INSTALACIONES

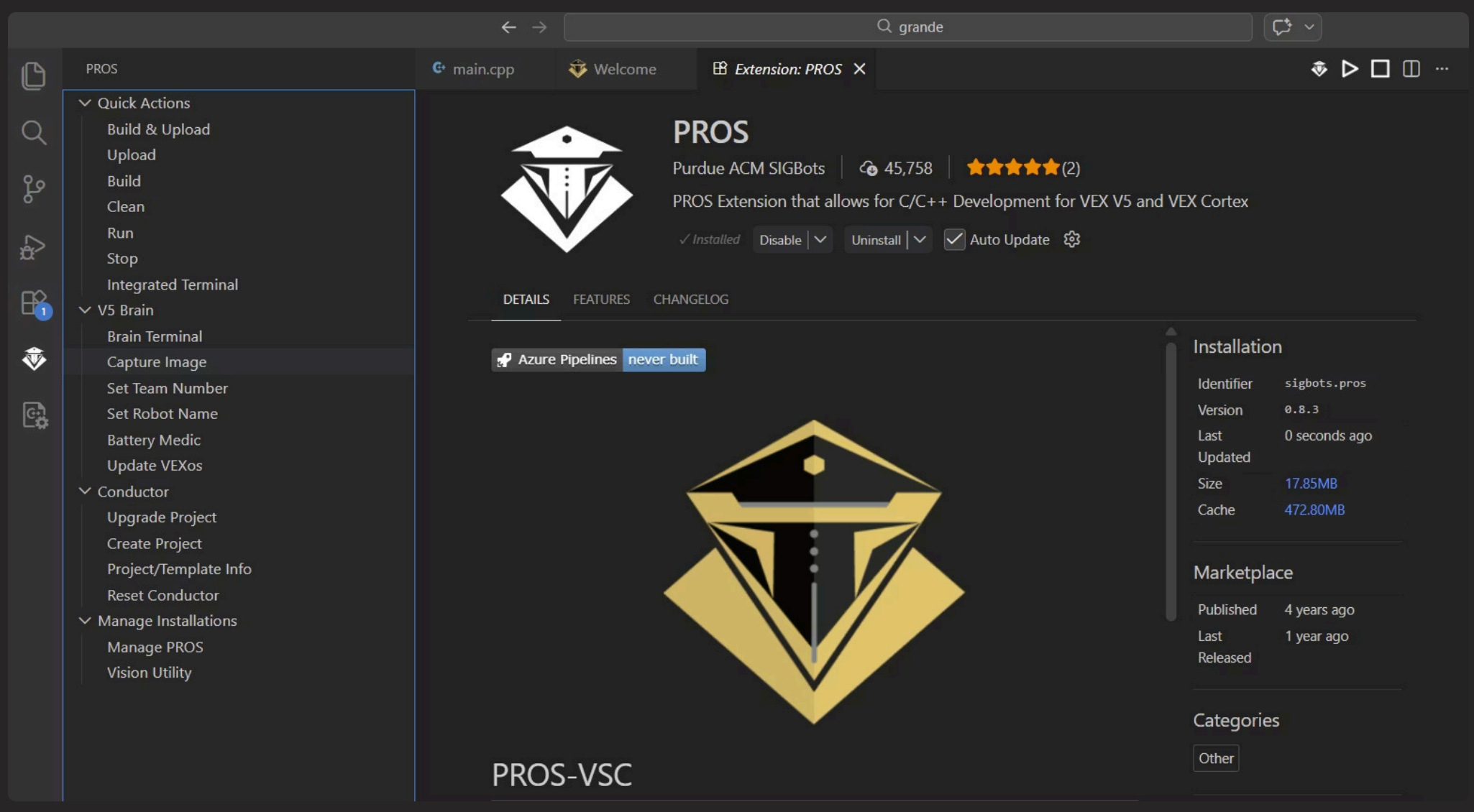
Descargar VS Code

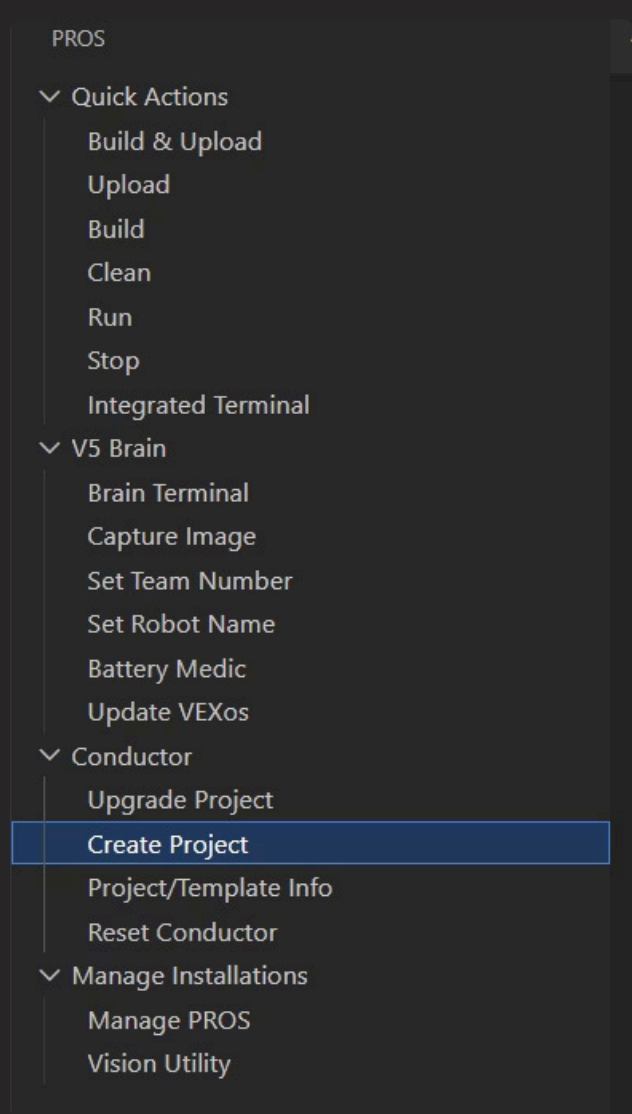


Abre extensiones y descarga PROS

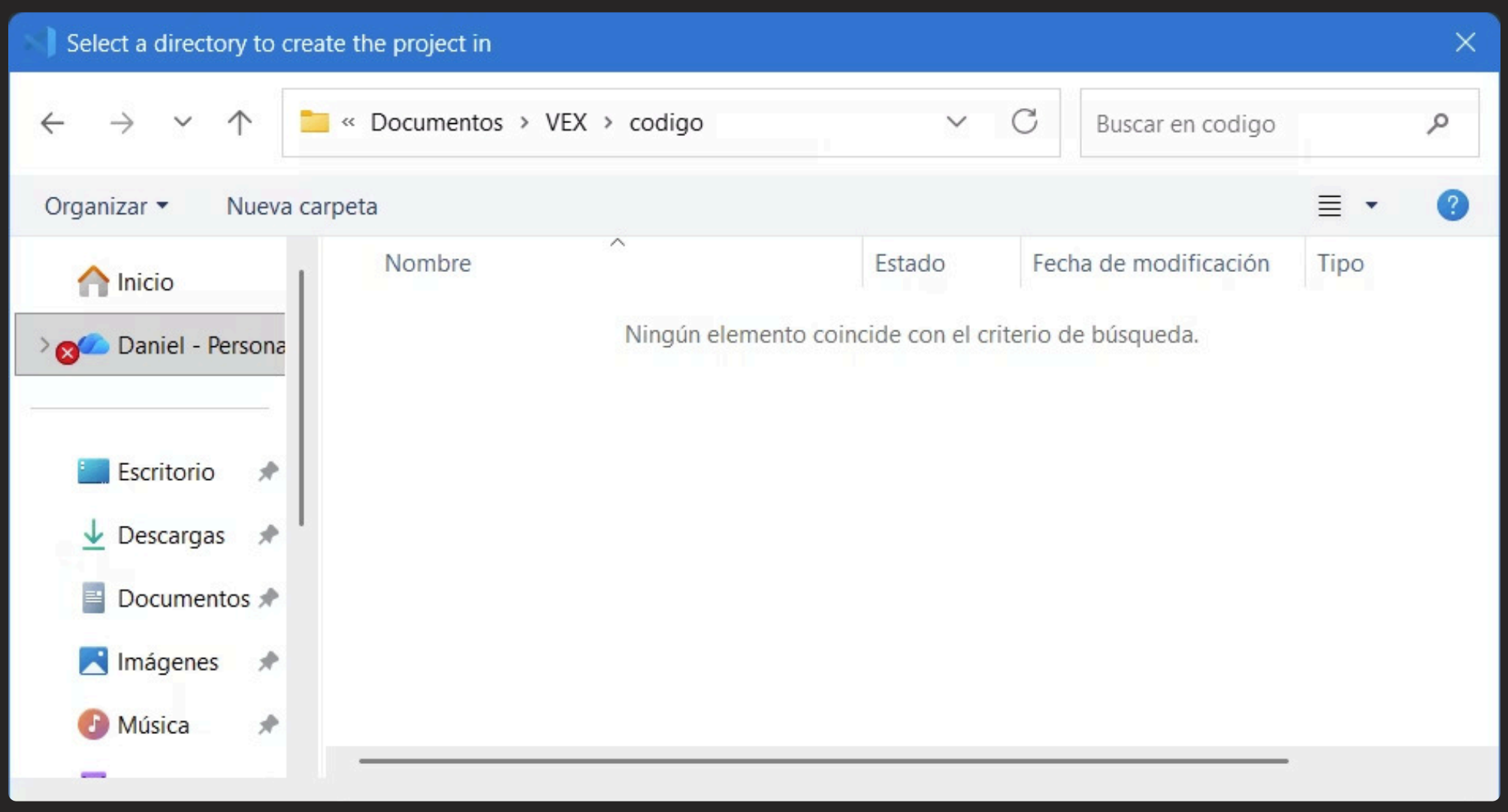


Made with GAMMA





CREATE PROJECT



CREAR CARPETA

DOCS»VEX/VORTEX»CODE

Made with GAMMA

test_robot

main.cpp

Enter a name for the project (Press 'Enter' to confirm or 'Escape' to cancel)

PROS

Purdue ACM SIGBots | 45,758 | (2)

PROS Extension that allows for C/C++ Development for VEX V5 and VEX Cortex

Select the target device

v5

- v5
- cortex

Select Project Version

latest

- latest Recommended
- 4.2.2
- 4.2.1
- 4.1.2
- 4.1.1
- 4.1.0
- 3.8.3

Made with GAMMA

EXPLORER

TEST_ROBOT

- .d
- .vscode
- bin
- firmware
- include
- src
  - main.cpp
- .gitignore
- common.mk
- compile_commands.json
- Makefile
- project.pros

Welcome | main.cpp

src > main.cpp > on_center_button()

```cpp
#include "main.h"

/**
 * A callback function for LLEMU's center button.
 *
 * When this callback is fired, it will toggle line 2 of the LCD text between
 * "I was pressed!" and nothing.
 */
void on_center_button() {
    static bool pressed = false;
    pressed = !pressed;
    if (pressed) {
        pros::lcd::set_text(2, "I was pressed!");
    } else {
        pros::lcd::clear_line(2);
    }
}

/**
 * Runs initialization code. This occurs as soon as the program is started.
 *
 * All other competition modes are blocked by initialize; it is recommended
 * to keep execution time for this mode under a few seconds.
 */
void initialize() {
    pros::lcd::initialize();
    pros::lcd::set_text(1, "Hello PROS User!");

    pros::lcd::register_btn1_cb(on_center_button);
}

/**
 * Runs while the robot is in the disabled state of Field Management System or
 * the VEX Competition Switch, following either autonomous or opcontrol. When
 * the robot is enabled, this task will exit.
 */
void disabled() {}

/**
 * Runs after initialize(), and before autonomous when connected to the Field
 * Management System or the VEX Competition Switch. This is intended for
 * competition-specific initialization routines, such as an autonomous selector
 * on the LCD.
```

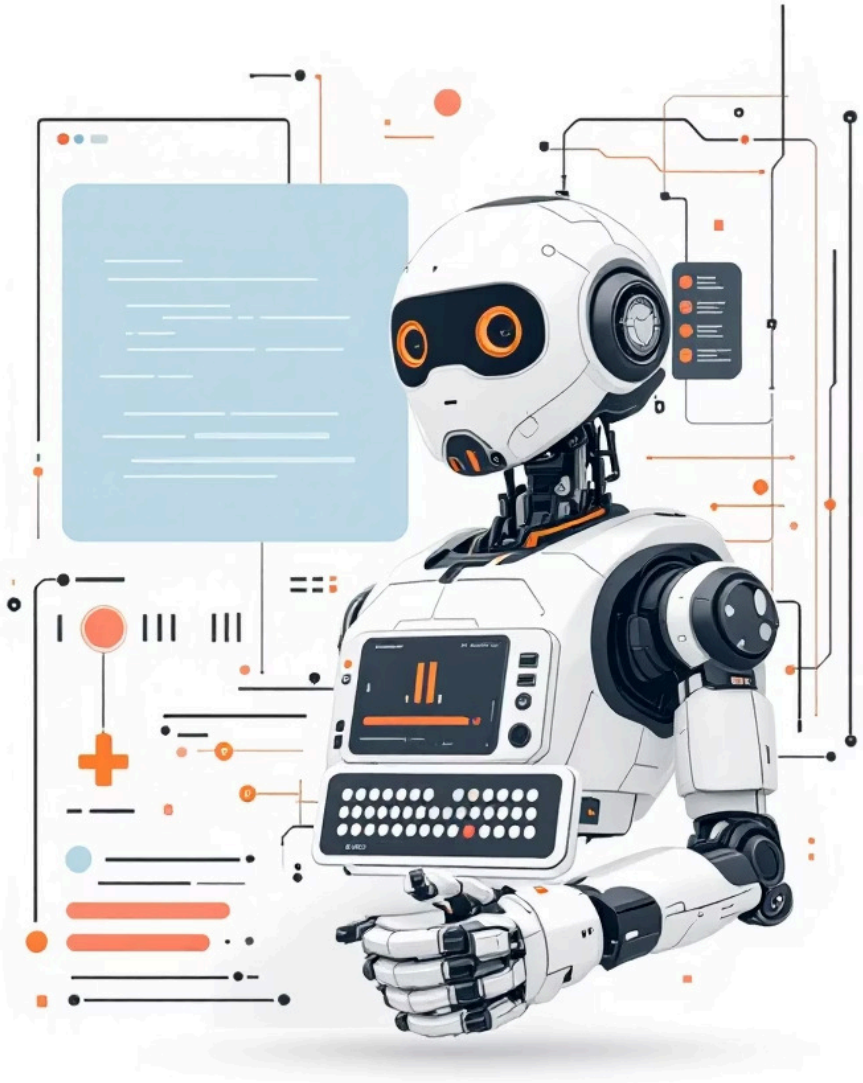
Made with GAMMA

Welcome | main.cpp

src > main.cpp > opcontrol()

```cpp
#include "main.h"

void initialize() {
    pros::lcd::initialize();
    pros::lcd::set_text(1, "Hello PROS User!");

}

void autonomous() {}


void opcontrol() {


    while (true) {

    }
}
```

Made with GAMMA



Hardware setup

DECLARAMOS MOTORES

```
pros::MotorGroup left_mg({8, 10});
pros::MotorGroup right_mg({1, 2});
```

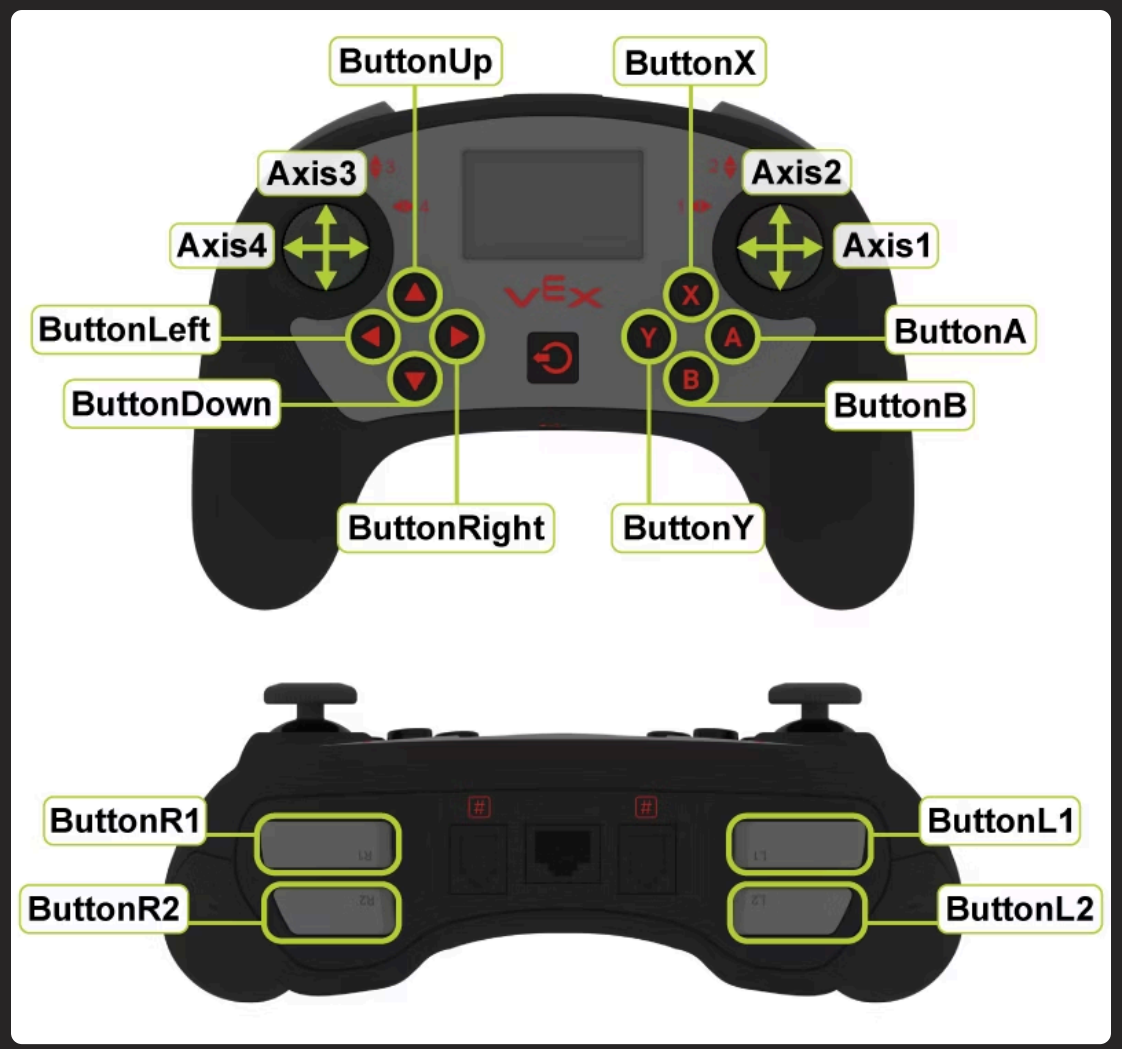
DECLARAMOS IMU

```
pros::Imu imu(5);
```

DECLARAMOS EL CONTROL

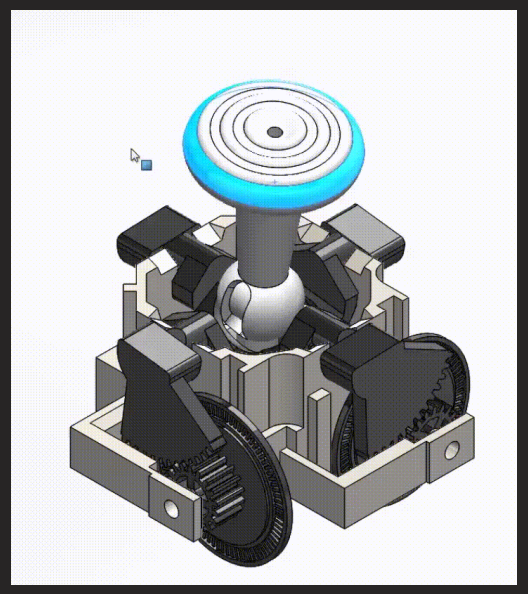
```
pros::Controller master(pros::E_CONTROLLER_MASTER); // entrada control
```

Made with GAMMA



Joysticks (analógico)

```
master.get_analog(pros::E_CONTROLLER_ANALOG_RIGHT_Y); // avance/retroceso
master.get_analog(pros::E_CONTROLLER_ANALOG_LEFT_X); // giro
```



~ 127 → arriba / derecha

~ -127 → abajo / izq

0 → centro

- RIGHT_Y → avanzar/retroceder
- LEFT_X → girar

Made with GAMMA

Control manual (opcontrol)

```
void opcontrol() {}
```

Repetir el code

siempre hasta que salgas de opcontrol()

```
void opcontrol() {
    while (true) {
    }
}
```

power (±) = avanzar + / retroceder -

Definimos el stick derecho y eje vertical Y

```
int power  =  master.get_analog(pros::E_CONTROLLER_ANALOG_LEFT_Y);  // adelante/atrás
```

girar

Definimos el stick izq y el eje horizontal

```
int turn = -master.get_analog(pros::E_CONTROLLER_ANALOG_RIGHT_X); // girar
```

deadband

evita accidentes

```
if (std::abs(turn) < 5) turn = 0;
if (std::abs(power)  < 5) turn  = 0;
```

Si el valor es menor, tratarlo como si fuera 0.

mezcla

Para girar un lado del robot debe de empujar el otro.

```
int left_power  = power + turn;
int right_power = power - turn;
```

Avanzar Recto:

```
power = 80
turn = 0
...
left_pwr = 80 + 0 = 80
right_pwr = 80 - 0 = 80
```

Girar:

```
power = 0
turn = -80
...
left_pwr = 0 + 80 = 80
right_pwr = 0 - 80 = -80
```

Mover los motores:

se mueve de acuerdo a los joysticks:

```
left_mg.move(left_power);
right_mg.move(right_power);
```

```
void opcontrol() {
        while (true) {

                int power = -master.get_analog(pros::E_CONTROLLER_ANALOG_RIGHT_Y); // adelante/atrás
                int turn  =  master.get_analog(pros::E_CONTROLLER_ANALOG_LEFT_X);  // girar

                // Deadband
                if (std::abs(power) < 5) power = 0;
                if (std::abs(turn)  < 5) turn  = 0;

                // Mezcla
                int left_power  = power + turn;
                int right_power = power - turn;

                // Mover motores
                left_mg.move(left_power);
                right_mg.move(right_power);

                pros::delay(20);

                printIMU();
                // printMotors();
        }
}
```

Made with GAMMA

NOTIFICATIONS

Device VEX V5 Controller Port (COM5) has disconnected!

Project Uploaded Successfully

Made with GAMMA

project.pros

PROS Project Settings

Upload: **Project Name**

This shows as the program's name on the brain when uploaded.

test_robot

Upload: **Project Description**

This shows as the program's description on the brain when uploaded.

My PROS Project

Upload: **Program Slot**

Set the Program Slot number that this project will be uploaded into by default on the V5 brain.

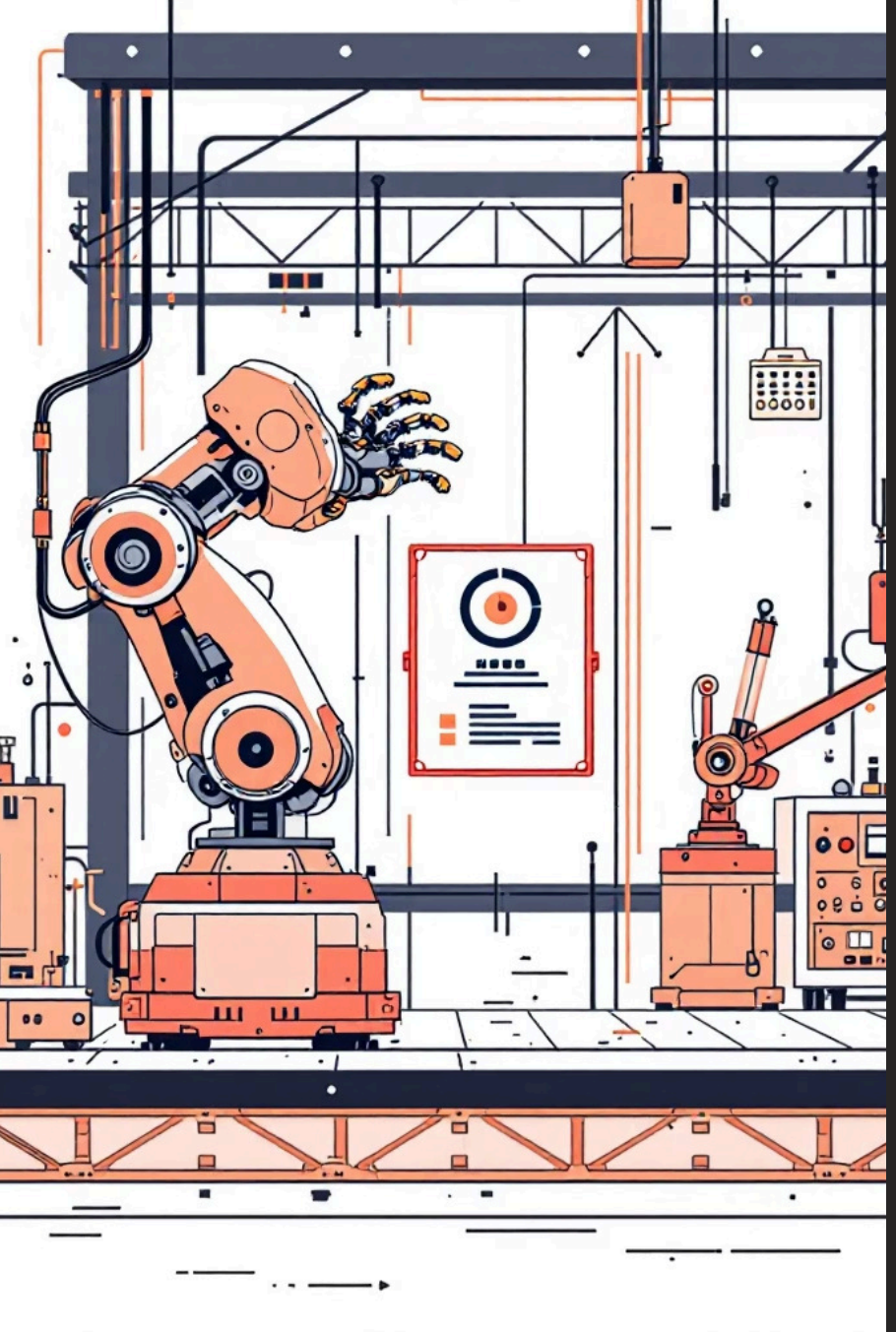
1

Upload: **Program Icon**

Set the Program Icon do be displayed on the V5 Brain.

PROS

Made with GAMMA



¿Por qué un robot falla aunque esté programado?

```
double error = target - imu.get_yaw(); // mide diferencia
double output = kp*error + ki*integral + kd*derivative; // calcula corrección
left.move(-output); // aplica movimiento
right.move(output);
```

El problema fundamental: Falta de control en tiempo real. El robot ejecuta comandos sin verificar si los está realizando correctamente.

Made with GAMMA

Sensores clave en VEX V5

IMU

Mide la orientación del robot.

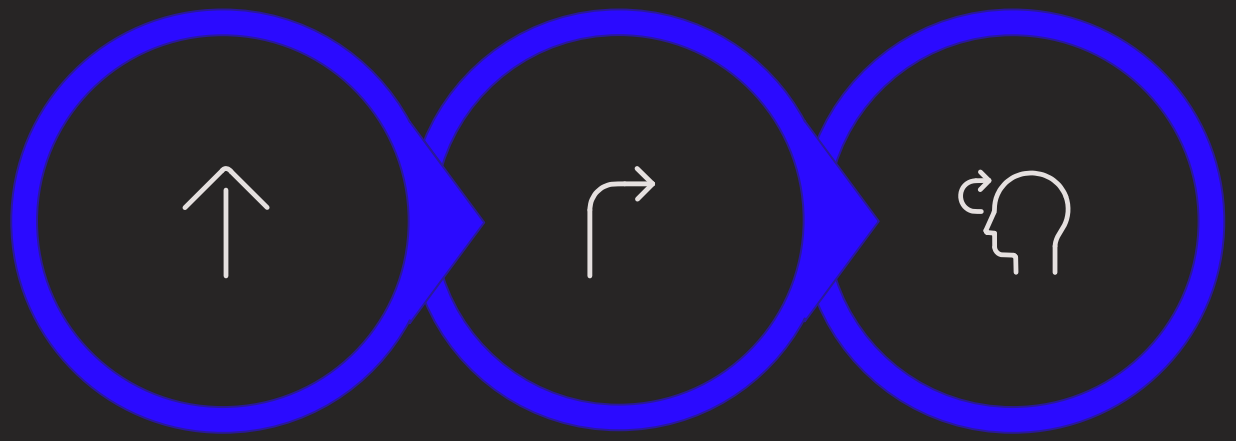
Encoders

Miden la distancia recorrida.

Sin sensores = robot ciego.

Made with GAMMA

IMU: Medición de orientación



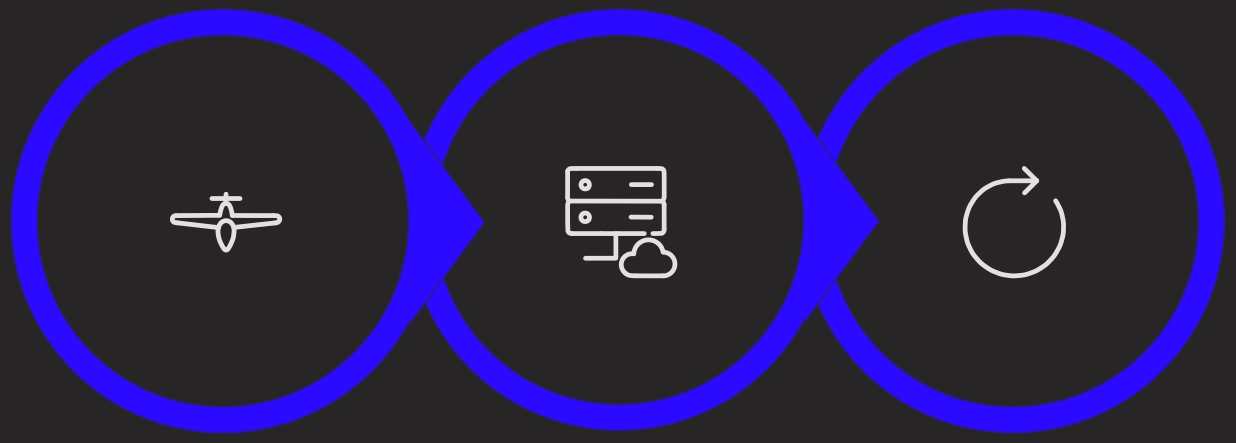
0° Adelante

90° Derecha

180° Atrás

Made with GAMMA

Encoders: Medición de movimiento



Motor girando

Obtener posición

360° = 1 rotación

Código

```
motor.get_position();
```

Concepto

Mide la rotación del eje en grados.

Ejemplo

360° = 1 rotación completa

Made with GAMMA

Print IMU

Declara el IMU

```
const int IMU_SIGN = 1;
```

Inicializamos y calibramos el IMU

```
void initialize() {
  pros::lcd::initialize();
  pros::lcd::set_text(1, "Hello PROS User!");

   imu.reset();
  while(imu.is_calibrating()){ pros::delay(10);}
}
```

```
    void printIMU() {
   static uint32_t last_print = 0;

   if (pros::millis() - last_print > 150) {
      last_print = pros::millis();

      int yaw = (int)(IMU_SIGN * imu.get_yaw());

      master.print(0, 0, "Yaw:%4d      ", yaw);
   }
}
```

Made with GAMMA

Print Motors

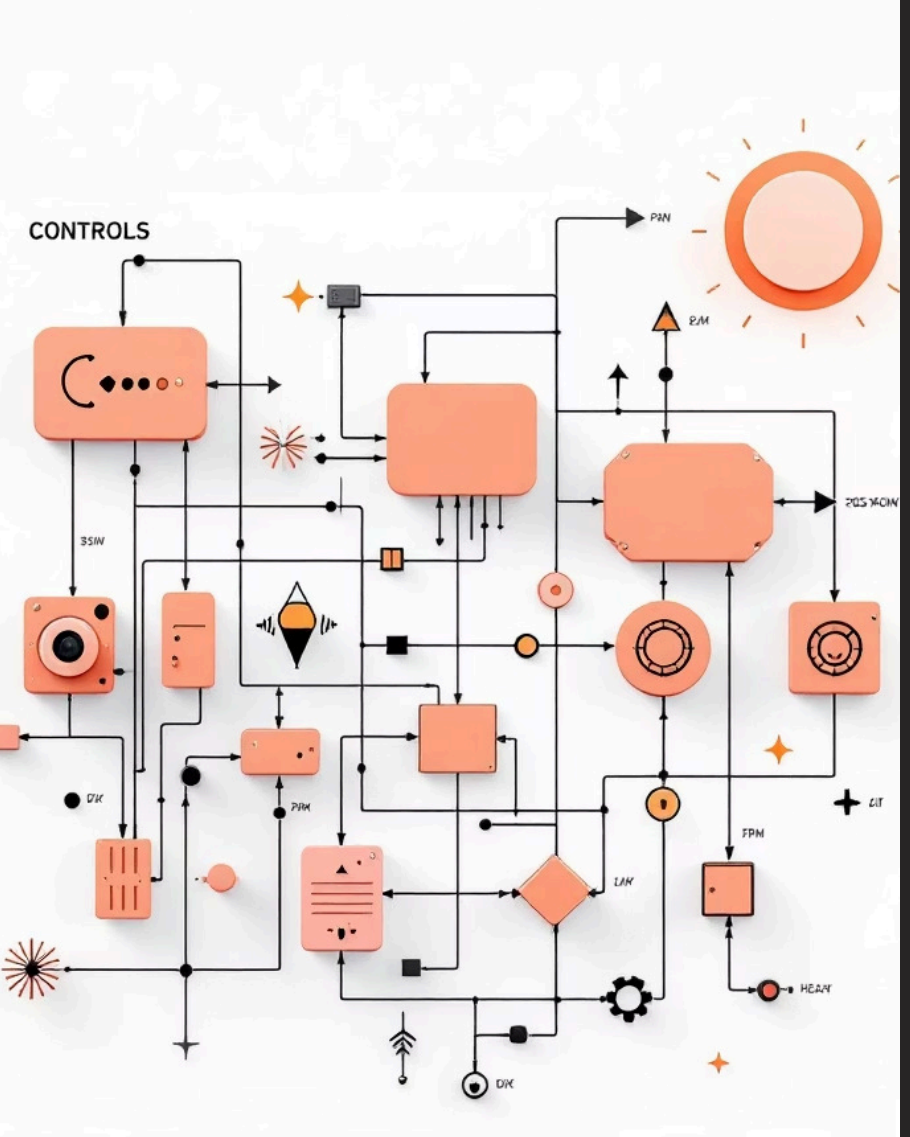
```
void printMotors() {
    static uint32_t last_print = 0;

    if (pros::millis() - last_print > 150) {
        last_print = pros::millis();

        int l_rpm = (int)left_mg.get_actual_velocity();
        int r_rpm = (int)right_mg.get_actual_velocity();

        master.print(1, 0, "L:%4d R:%4d  ", l_rpm, r_rpm);
    }
}
```

Made with GAMMA

Cómo llamarlas

```
printIMU();
printMotors();
```

Made with GAMMA



Un robot es un sistema de control

Sensor

Controlador

Actuador

Este ciclo se repite constantemente

Made with GAMMA

Conceptos fundamentales de control

Objetivo (Target)

Lo que queremos alcanzar.

Estado Actual (Actual)

Lo que está sucediendo ahora.

Error

La diferencia entre ambos.

`error = target - actual`

Made with GAMMA

Traducción simple

Sensor

¿Qué está pasando?

Error

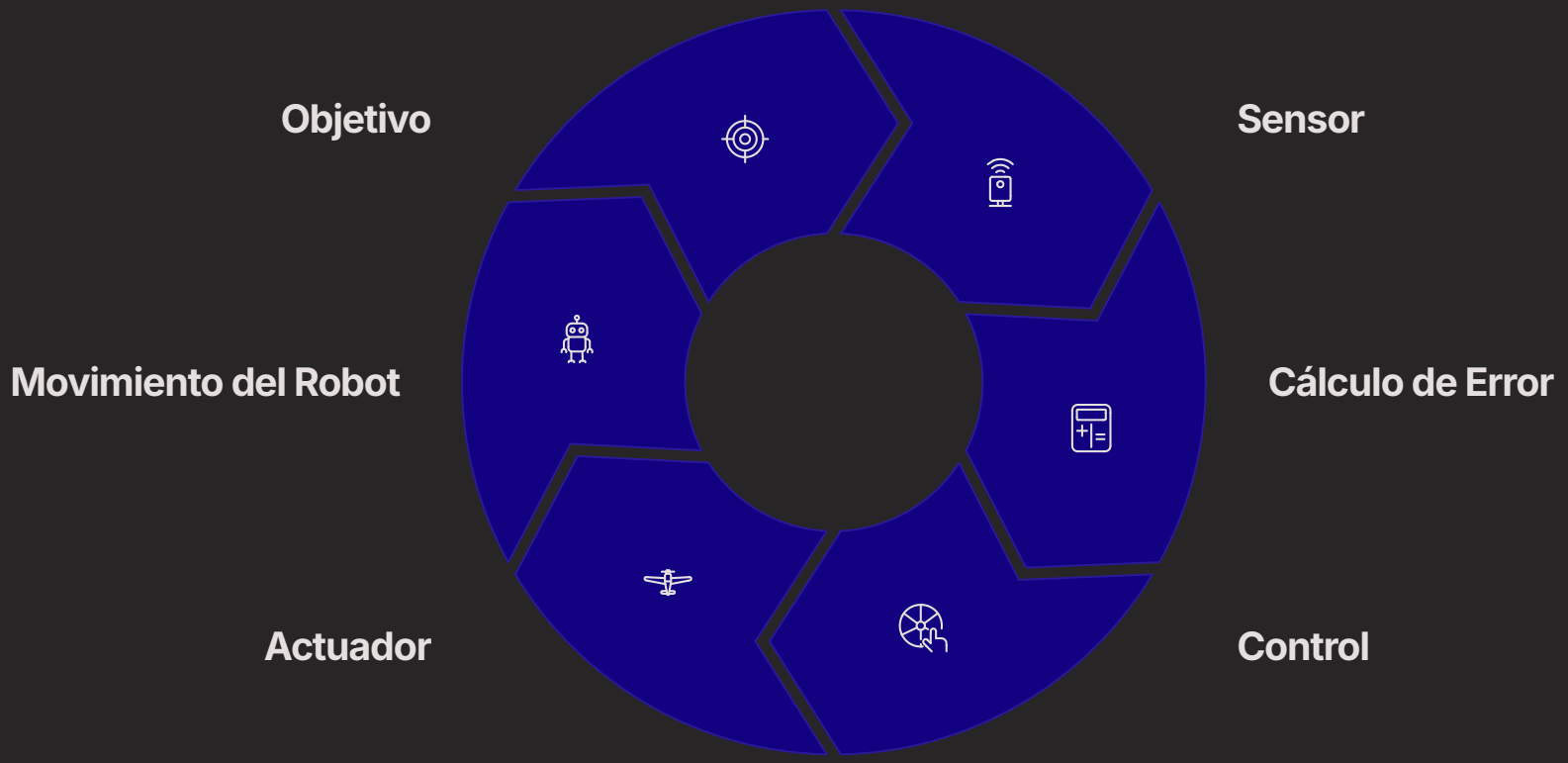
¿Qué tan lejos estoy?

Control

¿Qué hago para corregir?

Made with GAMMA

El ciclo completo del control

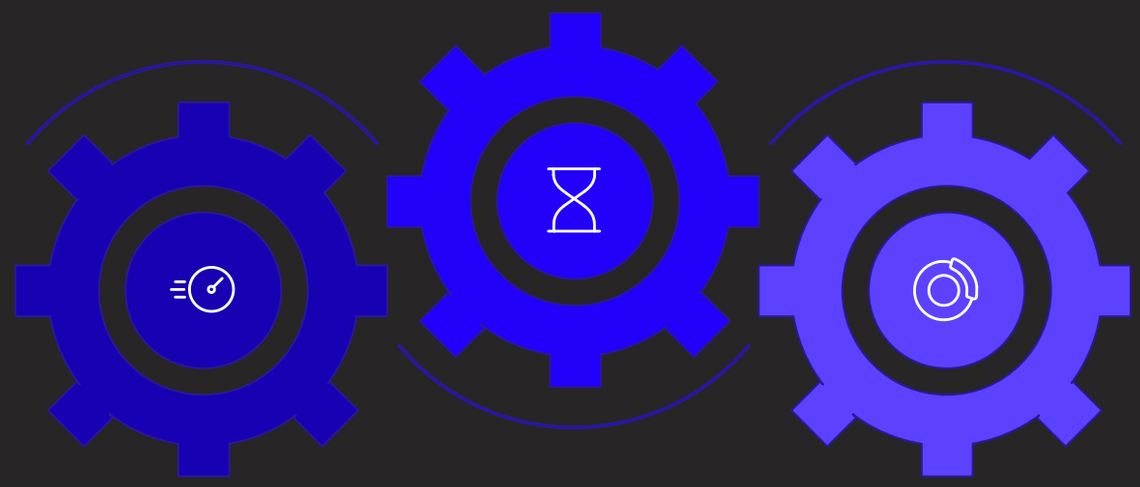


Made with GAMMA

PID: Todo junto 🔥

I - Integrador

Empuje constante por errores acumulados



P - Proporcional

Corrección fuerte para errores grandes

D - Derivativo

Freno para evitar el sobrepaso

P te acerca, I te ajusta, y D te estabiliza.

Made with GAMMA

GIRAR

Resetear el IMU:

```
imu.tare_yaw(); // empezar en 0°
```

Velocidad fija:

```
const int speed = 50;
```

Leer el angulo:

```
double yaw = imu.get_yaw();
```

Condición para detenerse:

```
if (yaw >= angleDeg) break;
```

Movimiento de giro

```
left_mg.move(-speed);
right_mg.move(speed);
```

Detener motores

```
left_mg.brake();
right_mg.brake();
```

```
void turn(double angleDeg) {

    imu.tare_yaw(); // empezar en 0°

    const int speed = 50;

    while (true) {

        double yaw = imu.get_yaw();

        // cuando llega al ángulo → se detiene
        if (yaw >= angleDeg) break;

        // girar
        left_mg.move(-speed);
        right_mg.move(speed);

        pros::delay(20);
    }

    left_mg.brake();
    right_mg.brake();
}
```

Made with GAMMA

Drive

Definimos una vel

```
int speed = 60;
```

Movemos os motores a esa vel

```
left_mg.move(speed);
right_mg.move(speed);
```

Mientras se cumplan los seg definidos

```
uint32_t start_time = pros::millis();
while ((pros::millis() - start_time) < time_ms) {
```

```
// ================== DRIVE SIMPLE ==================
void drive(double time_ms) {

    int speed = 60;

    uint32_t start_time = pros::millis();

    while ((pros::millis() - start_time) < time_ms) {

        left_mg.move(speed);
        right_mg.move(speed);

        pros::delay(20);
    }

    left_mg.brake();
    right_mg.brake();
}
```

Made with GAMMA

Con PID

Variables con PID

```
double integral = 0;
double last_error = 0;
```

- integral » acumula error
- last_error » guardar error anterior

Contantes

```
const double kp = 2.7;
const double ki = 0.01;
const double kd = 1.4;
```

- kp » que tan fuerte corrige
- ki » corrige errores acumulados
- kd » evita que se pase

```
const int max_speed = 60;
const int min_speed = 15;
const int tolerance = 1;
```

- max_speed » velocidad max
- min_speed » vel minima
- tolerance » que tan cerca es suficiente

Control del tiempo

Guarda el tiempo de inicio para que se ejecute por max 4 s

```
uint32_t start_time = pros::millis();
```

```
while ((pros::millis() - start_time) < 4000) {}
```

Medicion y error

lee el error

```
double yaw = IMU_SIGN * imu.get_yaw();
double error = angleDeg - yaw;
```

calcula el error

Condición de paro

si ya estás cerca » se detiene

```
if (std::abs(error) <= tolerance) break;
```

- si ya estás cerca → se detiene

INTEGRAL

```
integral += error; if (integral > 100) integral = 100;
if (integral < -100) integral = -100;
```

• acumula el error con el tiempo • limita para que no crezca demasiado

DERIVADA

```
double derivative = error - last_error; last_error = error;
```

• mide qué tan rápido cambia el error

PID

```
double output = (kp * error) + (ki * integral) + (kd * derivative);
```

• combina todo el PID

• P → corrige • I → ajusta • D → frena

LÍMITES

```
if (output > max_speed) output = max_speed;
if (output < -max_speed) output = -max_speed;
```

• evita que el robot vaya demasiado rápido

```
if (std::abs(output) < min_speed) output = (output > 0) ? min_speed : -min_speed;
```

• asegura que el robot sí se mueva

MOVIMIENTO

```
left_mg.move(-output); right_mg.move(output);
```

• un lado avanza, el otro retrocede

DELAY

```
pros::delay(20);
```

• espera 20 ms

FINAL

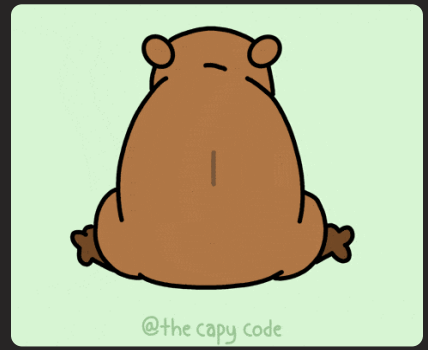
```
left_mg.brake(); right_mg.brake();
```

• detiene el robot al terminal

```
// ================= TURN =================
void turn(double angleDeg) {
  imu.tare_yaw();

  double integral = 0;
  double last_error = 0;

  const double kp = 2.7;
  const double ki = 0.01;
  const double kd = 1.4;

  const int max_speed = 60;
  const int min_speed = 15;
  const int tolerance = 1;

  uint32_t start_time = pros::millis();

  while ((pros::millis() - start_time) < 4000) {

    double yaw = IMU_SIGN * imu.get_yaw();
    double error = angleDeg - yaw;

    if (std::abs(error) <= tolerance) break;

    integral += error;
    if (integral > 100) integral = 100;
    if (integral < -100) integral = -100;

    double derivative = error - last_error;
    last_error = error;

    double output = (kp * error) +
                    (ki * integral) +
                    (kd * derivative);

    if (output > max_speed) output = max_speed;
    if (output < -max_speed) output = -max_speed;

    if (std::abs(output) < min_speed)
      output = (output > 0) ? min_speed : -min_speed;

    left_mg.move(-output);
    right_mg.move(output);

    pros::delay(20);
  }

  left_mg.brake();
  right_mg.brake();
}
```

Made with GAMMA

Se implementa




```
void autonomous() {
        turn(90);
        drive(2000); //avanza 2 seg
    }
```

Made with GAMMA

Tuning en la práctica 🎯

Ejemplo real:

Si mi robot gira a 90°

Solo P:

Se pasa a 100°, regresa a 80°, vibra

Con D:

Frena antes y llega más limpio

Con I mal ajustado:

Empieza a oscilar más

Por eso el tuning es importante. Cada robot es diferente.

Made with GAMMA